

Model Provenance Testing for Large Language Models

Ivica Nikolic^{*}
National University of Singapore
Singapore inikolic@nus.edu.sg

Teodora Baluta
Georgia Institute of Technology
Georgia, USA
teobaluta@gatech.edu

Prateek Saxena
National University of Singapore
Singapore prateeks@comp.nus.edu.sg

Abstract

Large language models are increasingly customized through fine-tuning and other adaptations, creating challenges in enforcing licensing terms and managing downstream impacts such as protecting intellectual property or identifying vulnerabilities. We address this challenge by developing a framework for testing *model provenance*. Our approach is based on the key observation that real-world model derivations preserve significant similarities in model outputs that can be detected through statistical analysis. Using only black-box access to models, we employ multiple hypothesis testing to compare model similarities against a baseline established by unrelated models. On two comprehensive real-world benchmarks spanning models from 30M to 4B parameters and comprising over 600 models, our tester achieves 90 – 95% precision and 80 – 90% recall in identifying derived models. These results demonstrate the viability of systematic provenance verification in production environments even when only API access is available.

1 Introduction

Platforms such as Amazon SageMaker and Hugging Face have enabled wide scale distribution of ML models, most notably large language models (LLMs) [39, 47, 22]. The increase of publicly available foundation models and datasets, however, has also triggered concerns over unauthorized use of intellectual property (IP) and concerns of compromised datasets [7] or models [20]. These issues are present not just in open-source ecosystems, but also for proprietary models that are hidden behind APIs [3, 32]. For instance, concerns about model stealing attacks wherein one can extract the model parameters even for production-level models are on the rise [8, 42, 33]. Similarly, there is growing concern that models may contain backdoors or vulnerabilities, making them susceptible to jailbreaking [20, 2, 59]. Despite best efforts to create a safe environment for the development of foundation models, there have already been instances of reported misuse [38, 16, 41].

This landscape highlights the growing need for *model provenance testing*. The problem of model provenance testing is as follows: Determine whether a target model has been derived from a foundational model by customizations such as fine-tuning. This problem has applications in tracking reuse of models not just in open marketplaces but also across product teams in large organizations.

When a security or privacy audit finds a problem with a foundational model, it becomes important to identify which other models in use by the organization may be derived from the problematic one and take appropriate remedial actions (e.g. revoke, retrain, or fortify) to mitigate the risk of further

39th Conference on Neural Information Processing Systems (NeurIPS 2025).

non-compliant use. Model provenance tracking is often useful after the fine-tuned model has been deployed, and when authentic ground truth is unavailable or unreliable.

One challenging aspect of designing a model provenance tester is achieving high accuracy. There is a cost associated with a provenance verdict. For instance, as a result of provenance tracking, a company may initiate legal action or investigation. For use cases within the same organization, developers might have to revoke the use of an existing model and even retrain from a clean parent model. False positives, i.e., the deployed LLM is wrongly flagged as a derivation of a problematic LLM, thus entail a downstream cost. At the same time, false negatives, i.e., not being able to flag that the LLM is customized from a problematic parent, also increase the risk of non-compliance. Therefore, we want a principled way to decide provenance and to make accuracy trade-offs.

Another challenge is that a practical provenance tool needs to have *minimal assumptions* to be readily usable in many post-deployment settings. We focus on techniques that do not change typical training and data pipelines, and can be integrated for current state-of-the-art LLMs. The tester is expected to only have *black-box query* access to the models and has no additional information, such as the training dataset, test set, or the algorithm used for training. We are not aware of any prior work addressing the question of model provenance testing systematically and in such practical setups.

Contributions.. In this paper, we design the first practical model provenance tester for LLMs that requires only query access. Our proposed techniques stem from a key empirical observation: The output distribution of fine-tuned models is often close to that of their parent model. This distance between a model and its true parent is typically smaller than that between the model and other unrelated models, making it possible to reliably trace back a derived model to the original parent. In order to keep assumptions to a minimum, we propose to employ the principled framework of statistical hypothesis testing. Specifically, we use black-box sampling and estimation to determine whether the distribution captured by the given model is close to that of the candidate parent. Such estimation can provide formal statistical significance measures, which can be used to check for the *null hypothesis*, i.e., the customized LLM is not close to the given parent model. Our approach is agnostic to the chosen metric for closeness and returns with the guarantee that the false positive rate is less than or equal to a user-specified threshold. We conduct an extensive empirical evaluation across two comprehensive benchmarks comprising over 600 models from Hugging Face, ranging from 30M to 4B model parameters and across diverse domains. Our tester achieves 90-95% precision and 80-90% recall in detecting model provenance, even with a limited number of queries.

2 Model Provenance Testing

Pretraining LLMs involves significant investment, requiring substantial computational resources costing millions of dollars in infrastructure and thousands of GPU hours. When Company A releases a pretrained LLM denoted as f , it employs specific licensing terms crucial for protecting this investment, maintaining competitive advantage, and controlling the model’s usage [32, 3]. Startup B might download f , perform only fine-tuning or other light customization (mixture-of-experts, prompt engineering), but claim to have pretrained their model g from scratch, thereby circumventing licensing requirements and misrepresenting company A’s work. In such cases, we want to be able to *determine if g is derived through fine-tuning of f* and resolve the *model provenance problem*.

We consider model provenance testing framework with *minimal assumptions* only, where the tester can query the models f, g on arbitrary prompts (e.g., through an API) and get responses. The tester has no access to the training datasets, embeddings, hyperparameters used by either company, or any information about potential modifications performed by Startup B. This mirrors real-world conditions where companies do not always disclose their training procedures, data sources, or modification techniques, making the provenance testing problem both practical and challenging.

Our proposed methods are evaluated for a *non-adaptive adversary*, i.e., the Startup B is not aware of the strategy of the model provenance tester. On the other hand, it is not clear if more advanced techniques for evasion of provenance detection deployed by *adaptive adversaries* can provide real benefit due to the randomness used in our tester (sampling from infinite set of prompts) and the fact that adversarial fine-tuning may come at the cost of reduced model performance on intended task.

2.1 Related Work

The problem of testing provenance is similar to designing schemes to prove ownership of a model. We provide more details on related work in Appendix A and summarize main approaches here.

Fingerprinting. Fingerprinting aims to create an identifier of the model for different downstream purposes such as IP protection. Most prior work require white-box access or additional knowledge such as intermediate LLM modules [55], access to training and testing datasets and changes to the fine-tuning [49]. One type of approaches detect copies or tampering by hashing the model weights [9, 48]. Other works aim to create fingerprints by estimating the decision boundary via adversarial examples [27, 50, 37]. However, adversarial-based approaches can still have a high false positive rate, i.e., two unrelated models have the same fingerprinting samples [6]. Follow-up work generates universal perturbations that can be added to samples to match the target model [27]. This can be prone to adversarial evasion [19]. Thus, recent work constructs a set of adversarial examples that are connected with each other and that characterize the decision boundary more robustly [50]. In addition to model fingerprinting via adversarial perturbations, there are works that focus on measuring the model similarity [19, 17, 11, 23, 14]. For example, one work selects samples with different prediction results across two groups of reference models [17]. This approach utilizes their pairwise correlation to identify target stolen image classification models. Other approaches also rely on the internal representations to compare models, without focusing on provenance. For instance, one work locally approximates the models under test with linear models and then computes the cosine distance between the weights of the linear model [23]. One recent work in black-box access requires specialized prompting and an additional model’s embeddings to identify models [34]. We highlight that these approaches do not focus on the question of model provenance as fingerprints may not be robust to fine-tuning and other customizations. Our tester also has minimal assumptions of blackbox access to only next-token predictions. While our work considers this measure of similarity, our approach returns with a bounded false positive rate, which is metric-agnostic.

Watermarking. Other approaches prove ownership via embedding watermarks in the model weights [10, 45, 43], hidden-layer activations [13] or other model features [26, 12]. Another type of watermarks are achieved by learning triggers or backdoors that produce predefined outputs [24, 40, 52, 36, 6, 56]. These rely on implementing changes to the white-box model, fine-tuning or training the model on a specialized dataset and can affect the model’s performance. Specialized model ownership schemes have also been proposed for other models such as graph neural networks [58, 44]. Most existing approaches do not have provable guarantees that model ownership can be verified with a given confidence, so the verification is often empirically determined. It is particularly challenging to preserve the watermark detection after applying model customizations [25].

The focus of prior work has not been on designing tests for determining provenance under model customizations and do not work under the same minimal assumptions. For instance, recent work proposed a detection framework for fine-tuning and pruning that still requires white-box access to

the victim model to generate test cases and it does not consider LLMs [11]. Moreover, our benchmarks are much more extensive with hundreds of models under diverse customizations.

3 Approach

Our approach to testing model provenance is based on a key observation: fine-tuning and other model derivation techniques typically result in only limited changes to the original model, as they primarily adapt the model for new tasks. After fine-tuning, the derived model g may remain similar to its parent model f , as the process focuses on refining specific capabilities rather than creating fundamental changes to the model distribution.

3.1 Model Provenance Tester

Since the tester has only query access to the models, the only data it can collect is from providing inputs (called *prompts*) and analyzing the corresponding output tokens. Furthermore, due to our minimal assumptions and lack of information about the training datasets, the tester queries the models on randomly chosen short sequences, further denoted as a prompt space Ω .

The tester independently samples from Ω a set of T prompts $x_1, \dots, x_T$, queries each model on the same set of prompts, and then compares their output tokens pairwise. For each prompt, we compare only the first output token generated by each model; however, n -grams could also be considered. The *similarity* between two models f and g is then calculated as the proportion μ of prompts on which the models produce the same output token: $\mu = \frac{1}{T} \sum_{j=1}^T \mathbb{1}(f(x_j) = g(x_j))$.

However, the ratio μ by itself may not be sufficient to determine whether f and g are similar, since even two unrelated models might agree on some proportion of outputs by chance. To assess the significance of μ , we introduce a set of *control models* $C = \{c_1, \dots, c_m\}$ for the model f and compare their similarities to g . These control models are sampled from the set of available LLMs with distinct architectures or training data, to ensure diversity. Our aim is to compare μ

The final step of the tester is to verify that the similarity ratio μ between f and g exceeds all similarity ratios μ_i . However, we want to ensure this is not merely due to random chance, but rather

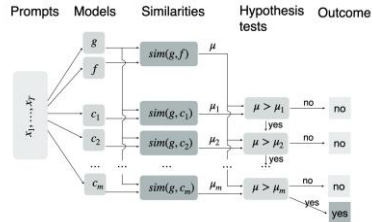
to the similarity ratios μ_i between each control model c_i and g . The number m of control models and their sampling strategy depend on availability; generally, the larger the set of control models, the better the estimation of similarity among unrelated models.

reflects a true difference. To establish such theoretical guarantees, we employ multiple hypothesis testing. More precisely, for each control model c_i , we formulate the following hypothesis test H^i :

$$H_0^i: \mu \leq \mu_i,$$

$$H_1^i: \mu > \mu_i,$$

Figure 1: Our model provenance tester that decides if model g is derived from model f .



where H_0^i is the null hypothesis that the similarity between f and g is less than or equal to the similarity between c_i and g , and H_1^i is the alternative hypothesis that the similarity between f and g is greater. To test each of the hypothesis H^i , we employ a z-test, which is a standard statistical test well-suited for comparing proportions like our similarity ratios when working with large samples. The z-test produces a p -value, which represents the probability of observing such a difference in proportions if there were truly no difference between the models (i.e., if the null hypothesis were true). A small p -value (typically less than a predetermined significance level α) indicates that the similarity μ between f and g is indeed significantly larger than the similarity μ_i between g and c_i .

When conducting multiple hypothesis tests simultaneously, we want to maintain this same overall risk level of α , regardless of how many tests we perform. However, running multiple tests increases our chances of obtaining at least one false positive across all tests, known as the family-wise error rate (FWER). To control this cumulative risk, we employ the Holm-Bonferroni method [18], which adjusts the significance thresholds α_k for individual tests H^k to ensure the overall false positive rate remains at or below our desired level of α . We illustrate this basic tester idea in Figure 1.

The provenance tester described thus far assumes we have one candidate parent f that we want to test our child model g against. We now extend it to the general case where the goal is to determine whether a model g is derived from some model from the set $f_1, \dots, f_s$ of candidate parent models. While running the basic tester s times (once for each provenance pair (f_i, g)) would solve the unspecified parent problem, it would also require additional correction for multiple testing to maintain the same level of confidence. The probability of false positives would grow with the number of candidate parents s unless appropriate adjustments are made to the significance level.

We thus propose the improved tester in Algorithm 1. Our tester avoids this issue by conducting a single set of hypothesis tests after identifying the most similar candidate. First, it finds the most similar model to the given model g among all the control models C and candidate parents F . If that model is a control model, the algorithm terminates with False. Otherwise, it goes on to test whether the FWER of this model is overall below α , the desired significance level. When this test procedure returns False, it indicates that we could not establish this higher similarity with the desired level of statistical significance. This may occur either because there is genuinely no significant similarity indicative of provenance or because the test lacked sufficient power under the given parameters (e.g., Algorithm 1 Provenance Tester for g Given a Candidate Parent Set

Require: Model g , candidate set $F = \{f_1, \dots, f_s\}$, set of control models $C = \{c_1, \dots, c_m\}$, prompt space Ω , number of prompts T , significance parameter α , statistical test $ZTest$.

```

 $x_1, \dots, x_T \sim \text{iid } \Omega$   $\triangleright$  Sample T prompts for
 $i \leftarrow 1$  to  $s$  do  $\triangleright$  Calc sim of candidates
 $\mu_i \leftarrow \frac{1}{T} \sum_{j=1}^T \mathbb{1}(f_i(x_j) = g(x_j))$ 
end for

for  $i \leftarrow 1$  to  $m$  do
 $\mu_i \leftarrow \frac{1}{T} \sum_{j=1}^T (c_i(x_j) = g(x_j))$   $\triangleright$  Calc sim of controls
end for

 $M \leftarrow \{\mu_1, \dots, \mu_s\} \cup \{\mu'_1, \dots, \mu'_m\}$   $\triangleright$  Set of all sims
 $\mu_{\max} \leftarrow \max(M)$   $\triangleright$  Find highest sim
if  $\mu_{\max} \in \{\mu_1, \dots, \mu_s\}$  then return FALSE  $\triangleright$  Highest not from  $F$ , but from  $C$ , so cannot be parent
end if

for  $\mu' \in M \setminus \{\mu_{\max}\}$  do
 $p_i \leftarrow ZTest(\mu_{\max}, \mu', T)$   $\triangleright$  Compare against other sims
end for

 $(p_{(1)}, \dots, p_{(s+m-1)}) \leftarrow \text{Sort}(p_1, \dots, p_{s+m-1})$ 
for  $k \leftarrow 1$  to  $s + m - 1$  do
 $\alpha_k \leftarrow \alpha / (s + m - k)$   $\triangleright$  Holm-Bonferroni adjustment
if  $p_{(k)} > \alpha_k$  then return FALSE
end if
end for

return (TRUE,  $\text{argmax}_{i \in [s]} \mu_i$ )  $\triangleright$  Return parent

```

sample size or number of prompts) to detect it. When the algorithm return True, it has the guarantee that the most similar model is one of the candidate models and that the total significance level across all hypotheses meets the threshold α .

3.2 Understanding Sources of Error

The overall error rates of our tester depend on the combination of errors from both our statistical hypothesis testing and the two core assumptions which we describe below.

Assumption 1: Derivation implies similarity. We assume that when model g is derived from f , they will exhibit above-average similarity in their outputs. This assumption leads to two potential types of errors: false negatives and false positives. False negatives occur when a derived model shows insufficient similarity to its parent. This can happen when a model customizer applies extensive modifications that significantly alter the model’s behavior. While resource constraints typically prevent such extreme modifications (as they would approach the cost of training from scratch), some legitimate derivation relationships may still go undetected. False positives arise when independently developed models exhibit high similarity. This typically happens when models are trained on similar datasets or designed for similar specialized tasks - for instance, two independent medical diagnosis models may produce very similar outputs due to their shared domain constraints.

Assumption 2: Control models establish a valid baseline. We assume our control models provide a reliable baseline for the similarity we should expect between unrelated models. Similarly, poor selection of control models can lead to two types of errors. False positives occur when our control models are too dissimilar from the domain of f . For example, using general language models as controls for specialized code generation models sets an artificially low baseline, making normal domain-specific similarities appear significant. False negatives happen when control models are themselves derived from f or trained on very similar data. This establishes an artificially high baseline that masks genuine derivation relationships.

While we can provide theoretical guarantees for controlling error rates in hypothesis testing, we cannot derive analytical bounds for errors arising from the assumptions about derivation implying similarity or the validity of the control model baseline. The assumption-based error rates can only be evaluated empirically. We find these hold in practice through extensive experiments (Section 4).

3.3 Reducing Query Complexity

Most of LLMs available currently allow cheap (even free) API access, thus the monetary query cost of running our testers is insignificant. However, sometimes this is not the case, for instance when the cost of queries is high (e.g. one query to OpenAI model O1 can cost more than \$1 [31]), or when the models have some rate restrictions. Furthermore, there are use cases when query complexity can be reduced without any side effects, thus it makes sense from optimization perspective.

We can divide the queries used in the tester into two distinctive types: *online queries* made to the tested child model g , and *offline queries* made to the parent model f (or models $f_1, \dots, f_s$) and to the control models $c_1, \dots, c_m$. We make this distinction for two reasons. First, often offline queries are much cheaper, as the potential parent models (and the control models as we will see in the Section 4) are well established, and available from multiple sources, thus they are usually cheaper or free. Second, in some use cases, we can reuse the offline queries to perform many provenance tests of different g_i . We analyze separately these two scenarios.

Reducing Online Complexity. Since our tester is fundamentally based on statistical hypothesis testing, any reduction in query complexity must be compensated by increasing the statistical power of individual queries. Rather than querying model g with T random prompts, we can strategically select a smaller set of $T' < T$ prompts that yield comparable statistical power for detecting model provenance¹. We achieve this through an informed sampling approach: instead of uniform sampling from Ω , we employ rejection sampling with an entropy-based selection criterion.

¹ iid

It means in Algorithm 1, instead of random sampling $x_1, \dots, x_T \sim \Omega$, the goal is to find set $x_1, \dots, x_{T'}$ from $x_1, \dots, x_T$ and F, C .

Specifically, to generate each prompt in T' , we sample k candidate prompts from Ω and select the one that maximizes the entropy of output tokens across all parent and control models. The selection criterion is dynamically weighted to favor prompts that have stronger discriminative power between similar models. While this approach introduces dependencies between the sampled prompts (so the theoretical guarantees of hypothesis testing used in Algorithm 1 do not carry over), our empirical results in Section 4.4 demonstrate its practical effectiveness. We detail our approach in Appendix H.

Reducing Offline Complexity. We further consider the case of reducing offline complexity in settings where offline queries cannot be reused. The key observation for reducing offline query complexity is that we may not need an equal number of queries to all parent/control models to identify the most similar one. If a particular parent model shows consistently higher similarity to g compared to other models, we might be able to confirm it as the top candidate with fewer queries to the clearly dissimilar models. The challenge lies in determining when we have sufficient statistical evidence to conclude that one model is significantly more similar than the others, while maintaining our desired confidence levels. This observation naturally leads us to formulate the problem as a Best Arm Identification (BAI) [5] problem in the Multi-Armed Bandit (MAB) setting. In this formulation, each parent or control model represents an “arm” of the bandit, and querying a model with a prompt corresponds to “pulling” that arm. The “reward” for each pull is the binary outcome indicating whether the model’s output matches that of the tested model g . The goal is to identify the arm (model) with the highest expected reward (similarity to g) while minimizing the total number of pulls (queries). We implement an optimization for our tester based on state-of-the-art BAI approaches (see Appendix I for details) and show its impact in Section 4.4.

4 Evaluation

We evaluate our proposed provenance testing approach experimentally and seek to answer the following research questions:

- (RQ1) How accurate is our provenance tester in practice and how does the number of prompts affect its performance?
- (RQ2) To what extent do derived models maintain similarity to their parents?
- (RQ3) How does the size and selection of control models impact the tester?
- (RQ4) How effective are the query reduction approaches?

4.1 Models and Provenance Testing Parameters

We collect model candidates for all provenance pairs from the Hugging Face (HF) platform [21]. To avoid selection bias, we used download counts as our selection criterion, taking the most popular models subject only to hardware constraints on model size.

To increase variety of candidates, we create two distinct benchmarks BENCH-A and BENCH-B, that differ in aspects such as model sizes, choice of pre-trained models, and ground-truth verification procedure (refer to Tbl. 1). The full procedure of collection of models and constructions of benchmarks is described in Appendix C. We use the standard significance $\alpha = 0.05$ (see Appendix G for other values). Sampling of prompts is given in Appendix D.

The implementation of the tester along with the two benchmarks can be found at https://github.com/ivicanikolicsg/model_provenance_testing.

Selection of control set. In all of our provenance tests, we use the complete set of available pre-trained models from the benchmark as control models - 10

Table 1: Comparison of BENCH-A
BENCH-B on different features.

Feature	BENCH-A	BENCH-B
pre-trained models	10	57
derived models	100	383
total models	100	531
model parameters	1B-4B	< 1B
compilation method	manual	automatic
ground-truth verification	higher	lower

models for BENCH-A and 57 for BENCH-B. This selection was done to demonstrate that effective control sets can be constructed without careful manual curation or domain-specific analysis. Manual curation would not have been feasible since our two benchmarks have over 600 candidates. Specifically, we make no effort to align control models with particular parent models’ domains or capabilities. We neither analyze the outputs of parent models f nor attempt to match control models to specific use cases. Instead, we simply include all pre-trained models that rank among the most popular on the Hugging Face platform. This sampling approach, while simple, helps avoid introducing obvious selection bias while ensuring broad coverage of model types and capabilities. Due to the large benchmark size, we find diverse examples of candidates for domains such as financial, medical and more (see Appendix C for details). We have not evaluated other selection strategies, which remains a direction for future work. This straightforward selection strategy, however, allows us to evaluate whether provenance testing can be effective even without carefully chosen control sets.

4.2 Accuracy of Model Provenance Tester

We evaluate the accuracy of the provenance tester by examining its performance on both BENCH-A and BENCH-B under different numbers of prompts. Figure 3 shows the precision and recall results from these experiments.

The precision is notably high (approximately 0.95) when the tester uses up to 1,000, and it is significantly higher than the baseline precision of randomly guessing the parent which is $\frac{1}{10+1} \approx 0.09$

for BENCH-A and $\frac{1}{57+1} \approx 0.02$ for BENCH-B. Interestingly, however, the precision reduces as the number of prompts (test samples) increases. This is in direct contrast to common hypothesis testing, where larger sample size leads to smaller standard errors, thus higher precision. We get different results because our model provenance tester relies on detecting similarities of models. When using a smaller number of prompts, it can detect only the stronger similarities which are usually due to model provenance. However, as we increase the prompts, it starts detecting similar models that not necessarily have provenance relation. This leads to misclassification and reduced precision.

The recall behavior shows an opposite trend - it improves with a larger number of prompts, eventually reaching 80% – 90% depending on the benchmark. This follows expected behavior: more prompts increase the statistical power of our hypothesis tests, enabling detection of small but significant differences in similarities. This increased sensitivity leads to higher recall rates, as the tester can detect more subtle provenance relationships that might be missed with fewer prompts.

We also examine the impact the randomness of prompt sampling on the tester’s accuracy. We conduct experiments on both benchmarks using five different randomly sampled sets of 1,000 prompts, with the same set of prompts used in all testers, and record the precision and recall for each run – see Table 6 of Appendix E. The results show that these values vary by 1 – 4% between runs, indicating consistent performance across different prompt samples.

(RQ1): Our model provenance tester demonstrates high accuracy across different benchmarks, achieving precision of 90%–95% and recall of 80%–90% with 3,000 prompts per model. Simply increasing the number of prompts does not guarantee uniformly better results, reflecting a fundamental trade-off: gains in recall might be accompanied by losses in precision.

4.3 Correctness of Assumptions

As discussed in Section 3.2, our approach relies on two key assumptions. While the high accuracy demonstrated in the previous section indirectly validates these assumptions, we provide here a detailed experimental analysis of both.

Our first assumption posits that derived models maintain significant similarity to their parent models. To evaluate this, we analyzed the similarity rankings across all provenance tests using 3,000 prompts. For each derived model, we examined where its true parent ranked among all models in terms of similarity ratio μ . The results strongly support this: in BENCH-A, the true parent had the highest similarity

indicate that derived models do indeed maintain strong similarity patterns with their parent models. Inadvertently, we have shown as well that with 3,000 prompts the tester almost approaches the statistical limit (only the model with highest similarity ratio can be identified as a parent), as the recalls are very close to the per-ratio in 93% of cases, while in BENCH-B this occurred in 89% of cases. When considering whether parents ranked in the top 50th percentile by similarity, these percentages increased to 98% and 96% respectively. Thus we can conclude that our experiments centages of highest similarity (89% recall vs. 93% highest parent similarity, and 82% recall vs. 89% similarity, for the two benchmarks, respectively).

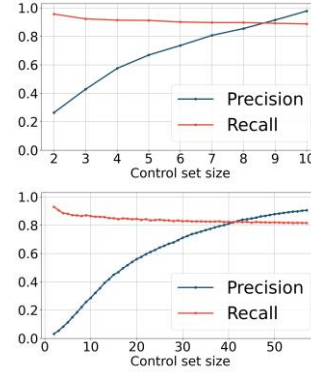


Figure 2: Precision and recall of BENCH-B (left) and BENCH-A (right) with respect to smaller control set size.

(RQ2): The assumption of similarity between derived and parent models is valid for most provenance pairs.

Our second assumption concerns whether control models can effectively establish a baseline for similarity between unrelated models. We stress that in our experiments we have chosen the control models to be simply the set of all pre-trained models in an unbiased way, without any special selection or optimization for particular parent models they are tested against. We empirically observe that such unbiased selection of control model establishes a good baseline similarity as evident from the accuracy results presented thus far.

We further examine how the size and quality of the set of control models might affect tester accuracy. We conducted experiments varying the size of the control set while keeping other parameters constant (3,000 prompts per test). We randomly sampled different-sized subsets from our full control sets (10 models for BENCH-A and 57 for BENCH-B) and ran 100 complete benchmark tests for each size, and averaged the outcomes. The results, shown in Figure 2, reveal that both size and quality of the control set significantly impact tester performance. For BENCH-A, even with just 4 control models, the tester achieved 55% precision. This relatively good performance with few controls can

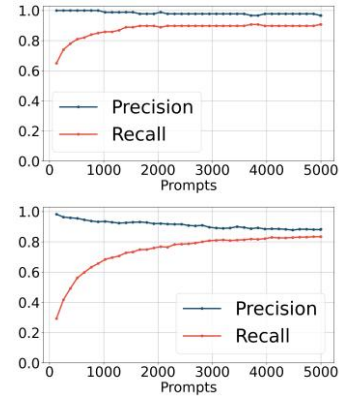


Figure 3: Precision and recall of the model provenance tester with different number of prompts on BENCH-A (top) and BENCH-B (bottom).

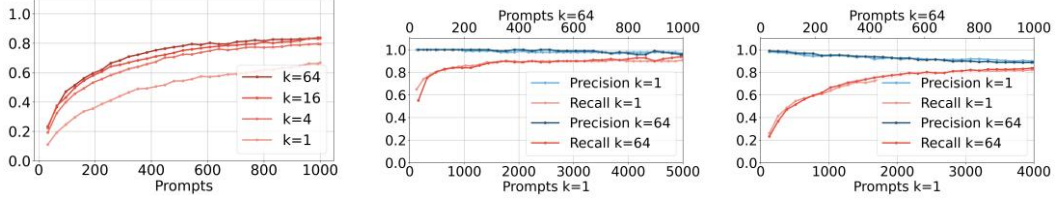


Figure 4: Recall for BENCH-B Figure 5: Precision/recall for BENCH-B (left) and BENCH-B with different values of advanced (right) when advanced online prompt sampling with $k = 64$ uses prompt sampling (k). four times less prompts than no advanced sampling ($k = 1$).

be attributed to BENCH-A consisting entirely of general-purpose, well-trained LLMs - thus any subset of these models provides a reasonable baseline for measuring similarity between unrelated models. However, for BENCH-B, the randomly sampled 4-model control set yielded less than 10% precision. This poor performance stems from BENCH-B containing a much more diverse set of models, including domain-specific models (e.g., for medical or coding tasks) and smaller models with varying capabilities. With such diversity, a small random subset of control models is unlikely to establish good baselines for all test cases - for instance, when testing a coding-focused model, we need coding-related models in the control set to establish proper baselines². Performance improves steadily as control set size increases in both benchmarks, since larger control sets are more likely to include appropriate baseline models for each test case.

(RQ3): The tester’s performance degrades when the control set is too small or poorly selected.

4.4 Reducing Query Complexity

To reduce the online complexity, we implement an advanced rejection prompt sampling strategy as detailed in Section 3.3. We evaluate this strategy using different parameter values $k = 4, 16$, and 64 (recall, k defines how many random samples are used to produce one selected sample), comparing it to the standard provenance testing without rejection ($k = 1$).

Figure 4 compares the tester’s recall across different values of k . Notable improvements are visible even at $k = 4$, with higher values of k showing better results (though with diminishing returns). Specifically, the recall achieved with 1,000 prompts at $k = 1$ can be matched using only about 250 prompts at $k = 64$, representing a four-fold reduction in online complexity. Figure 5 provides a comprehensive comparison between $k = 1$ and $k = 64$ for both precision and recall across both benchmarks, using 4 – 5 times fewer queries for $k = 64$ (note, in Figure 5 the number of prompts for $k = 64$ are given at the top of the plots). The results demonstrate that the tester maintains its effectiveness despite the significant reduction in queries to the tested models. For example, advanced prompt sampling achieves high levels of 90 – 95% precision and 80 – 90% recall while reducing the

Table 2: Precision and recall of the base vs BAI tester on BENCH-A and BENCH-B.

Allowed Queries T	Benchmark	Tester	Avg Queries	Precision	Recall
500	BENCH-A	base	500	1.00	0.81
500	BENCH-A	BAI	450	0.98	0.29
500	BENCH-B	base	500	0.95	0.56
500	BENCH-B	BAI	452	0.98	0.29
1,000	BENCH-A	base	1,000	0.99	0.86
1,000	BENCH-A	BAI	605	1.00	0.63
1,000	BENCH-B	base	1,000	0.94	0.68
1,000	BENCH-B	BAI	809	0.98	0.42
2,000	BENCH-A	base	2,000	0.98	0.89
2,000	BENCH-A	BAI	1,482	0.97	0.54
2,000	BENCH-B	base	2,000	0.92	0.77
2,000	BENCH-B	BAI	1,482	0.97	0.54

² Note that in practice, unlike our random sampling experiments, one can deliberately select control models matching the domain and capabilities of the suspected parent model, thus reducing significantly the impact of size of control sets, and leaving quality of the control set as the main factor on efficiency of the tester.

required number of prompts from 3,000 to just 500 per model.

We next evaluate strategies for reducing offline complexity, which refers to the number of queries made to pre-trained models during testing. We implement this reduction using BAI, as described in Section 3.3 and given in Algorithm 4. We test this approach on both benchmarks by setting a target budget of T queries (prompts) per pre-trained model. For example, with $T = 1000$ on BENCH-A, which contains 10 pre-trained models, the BAI-based provenance tester has a maximum budget of $10 \cdot 1,000 = 10,000$ total queries to make its decision.

Table 2 compares the performance of the base tester and the BAI-enhanced version across different query budgets $T \in \{500, 1000, 2000\}$. The results show that the BAI tester successfully reduces offline complexity by 10% – 30% (as shown in the “avg queries” column). However, this reduction comes at a significant cost to recall, while precision remains largely unchanged. For instance, with $T = 1,000$ on BENCH-A, BAI reduces the average number of queries from 1,000 to 605, but recall drops from 0.86 to 0.63. Similarly, on BENCH-B, the average queries decrease from 1,000 to 809, but recall falls from 0.68 to 0.42. This pattern persists across different values of T and both benchmarks, suggesting that the trade-off between query reduction and recall preservation is not favorable in most cases.

(RQ4): The online query optimization strategy leads to a 4-6 fold query reduction without accuracy drop, whereas the offline approach performs only marginally better and has a negative impact on recall.

5 Conclusion

Our work formulates the model provenance testing problem for LLMs which has many applications such as in detection of misuse of terms of use or vulnerable customized models. We present an approach based on statistical testing with minimal assumptions that has high accuracy for real-world benchmarks. Our key insight is that models derived through standard customization approaches maintain a level of similarity to their parent model that is statistically distinguishable from unrelated parents. We evaluate this observation empirically, together with our approach and several optimizations. We find our proposed method to be practical for deciding LLM provenance.

Acknowledgments

We thank Xu Louise, Bo Wang, Jason Zhijingcheng Yu, and Mallika Prabhakar for their valuable feedback on the draft. This research is supported by a Singapore Ministry of Education (MOE) Tier 2 grant MOE-T2EP20124-0007. This material is based upon work supported by the U.S. National Science Foundation under Grant No. 2531010. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the NSF.

References

- [1] Saeif Alhazbi, Ahmed Mohamed Hussain, Gabriele Oligeri, and Panos Papadimitratos. Llm’s have rhythm: Fingerprinting large language models using inter-token times and network traffic analysis. *arXiv preprint arXiv:2502.20589*, 2025.
- [2] Cem Anil, Esin Durmus, Nina Rimskey, Mrinank Sharma, Joe Benton, Sandipan Kundu, Joshua Batson, Meg Tong, Jesse Mu, Daniel J Ford, et al. Many-shot jailbreaking. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems (NeurIPS)*, 2024.
- [3] Rohan Anil, Andrew M Dai, Orhan Firat, Melvin Johnson, Dmitry Lepikhin, Alexandre Passos, Siamak Shakeri, Emanuel Taropa, Paige Bailey, Zhifeng Chen, et al. Palm 2 technical report. *arXiv preprint arXiv:2305.10403*, 2023.

- [4] Anonymous. Invisible traces: Using hybrid fingerprinting to identify underlying LLMs in genAI apps, 2025.
- [5] Jean-Yves Audibert and Sebastien Bubeck. Best arm identification in multi-armed bandits. In *Conference on Learning Theory (COLT)*, pages 13–p, 2010.
- [6] Xiaoyu Cao, Jinyuan Jia, and Neil Zhenqiang Gong. IPGuard: Protecting intellectual property of deep neural networks via fingerprinting the classification boundary. In *Proceedings of the 2021 ACM Asia Conference on Computer and Communications Security (AsiaCCS)*, pages 14–25, 2021.
- [7] Nicholas Carlini, Matthew Jagielski, Christopher A Choquette-Choo, Daniel Paleka, Will Pearce, Hyrum Anderson, Andreas Terzis, Kurt Thomas, and Florian Tramer. Poisoning web-scale training datasets is practical. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 407–425, 2024.
- [8] Nicholas Carlini, Daniel Paleka, Krishnamurthy Dj Dvijotham, Thomas Steinke, Jonathan Hayase, A Feder Cooper, Katherine Lee, Matthew Jagielski, Milad Nasr, Arthur Conmy, et al. Stealing part of a production language model. *arXiv preprint arXiv:2403.06634*, 2024.
- [9] Haozhe Chen, Hang Zhou, Jie Zhang, Dongdong Chen, Weiming Zhang, Kejiang Chen, Gang Hua, and Nenghai Yu. Perceptual hashing of deep convolutional neural networks for model copy detection. *ACM Transactions on Multimedia Computing, Communications and Applications*, 19(3):1–20, 2023.
- [10] Huili Chen, Bitar Darvish Rouhani, Cheng Fu, Jishen Zhao, and Farinaz Koushanfar. Deepmarks: A secure fingerprinting framework for digital rights management of deep learning models. In *Proceedings of the 2019 on International Conference on Multimedia Retrieval*, pages 105–113, 2019.
- [11] Jialuo Chen, Jingyi Wang, Tinglan Peng, Youcheng Sun, Peng Cheng, Shouling Ji, Xingjun Ma, Bo Li, and Dawn Song. Copy, right? a testing framework for copyright protection of deep learning models. In *2022 IEEE symposium on security and privacy (SP)*, pages 824–841, 2022.
- [12] Xuxi Chen, Tianlong Chen, Zhenyu Zhang, and Zhangyang Wang. You are caught stealing my winning lottery ticket! making a lottery ticket claim its ownership. *Advances in neural information processing systems*, 34:1780–1791, 2021.
- [13] Bitar Darvish Rouhani, Huili Chen, and Farinaz Koushanfar. Deepsigns: An end-to-end watermarking framework for ownership protection of deep neural networks. In *Proceedings of the twenty-fourth international conference on architectural support for programming languages and operating systems*, pages 485–497, 2019.
- [14] Tian Dong, Shaofeng Li, Guoxing Chen, Minhui Xue, Haojin Zhu, and Zhen Liu. Responsible identity audit governing the artificial intelligence. In *NDSS*, 2023.
- [15] Eyal Even-Dar, Shie Mannor, Yishay Mansour, and Sridhar Mahadevan. Action elimination and stopping conditions for the multi-armed bandit and reinforcement learning problems. *Journal of machine learning research*, 7(6), 2006.